

CSE 344 – System Programming

Homework #6

Magic Academy

Name: Ömer Zelka
Student ID: 230104004020
Course: CSE 344 – System Programming
Date: May 20, 2026

Contents

1	System Design	2
1.1	Overview	2
1.2	Single-Threaded Event Loop with <code>select()</code>	2
1.3	Idle Timeout Mechanism	3
1.4	Maximum Client Enforcement	4
2	Data Structures	4
2.1	Ingredient Storage	4
2.2	Per-Client State (<code>client_t</code>)	4
2.3	Wizard Spellbook	5
3	Partial Read/Write Handling	5
3.1	Partial Read Handling	5
3.2	Partial Write Handling	6
4	Signal Handling and Shutdown	6
4.1	Server Shutdown (Ctrl+C)	6
4.2	Client Exit (Ctrl+C)	7
5	Error Handling Summary	8
6	Compilation and Execution	8
7	Test Scenarios	8
7.1	Scenario 1: Error Cases	8
7.2	Scenario 2: Normal Wizard and Professor Operations	9
7.3	Scenario 3: Max Clients and Dynamic Connections	10
7.4	Scenario 4: Idle Timeout and Server Shutdown	10
8	Conclusion	16

1 System Design

1.1 Overview

The system consists of three separate programs communicating over TCP sockets:

- **hogwarts** – The single-threaded server that manages all client connections, ingredient inventory, and wizard spellbooks using I/O multiplexing.
- **wizard** – A client that connects to the server to submit BREW and CONSUME orders and maintain a personal spellbook.
- **professor** – A client that connects to the server to query ingredient quantities and connected client information.

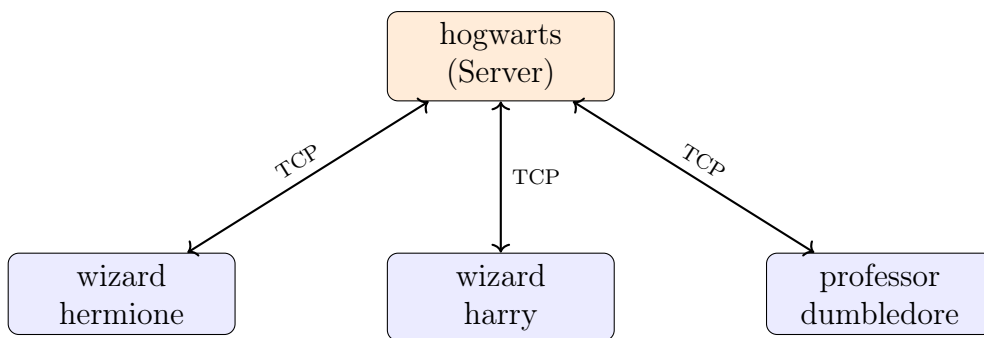


Figure 1: System architecture: multiple clients connect to a single-threaded server via TCP.

1.2 Single-Threaded Event Loop with `select()`

The server is strictly single-threaded. No threads or processes are spawned for individual clients. Instead, the `select()` system call is used to multiplex all file descriptors (the listening socket and all connected client sockets) within a single loop.

Each iteration of the main loop performs the following steps:

1. **Check idle timeouts:** Iterate over all connected clients and disconnect any client whose elapsed idle time exceeds the configured `TIMEOUT` value.
2. **Build `fd_set`:** Add the server listening socket and all active client sockets to the read set. Compute the minimum remaining timeout among all clients.
3. **Call `select()`:** Block until data arrives on any socket or the computed timeout expires. The timeout parameter ensures idle checking occurs without `sleep()` or threads.
4. **Accept new connections:** If the listening socket is readable, accept the incoming connection. If the server is at maximum capacity, send `ERR HOGWARTS_FULL` and close the socket immediately.
5. **Process client data:** For each client socket that is readable, read incoming data into the per-client line buffer. When a complete line (terminated by `\n`) is assembled, parse and execute the command.

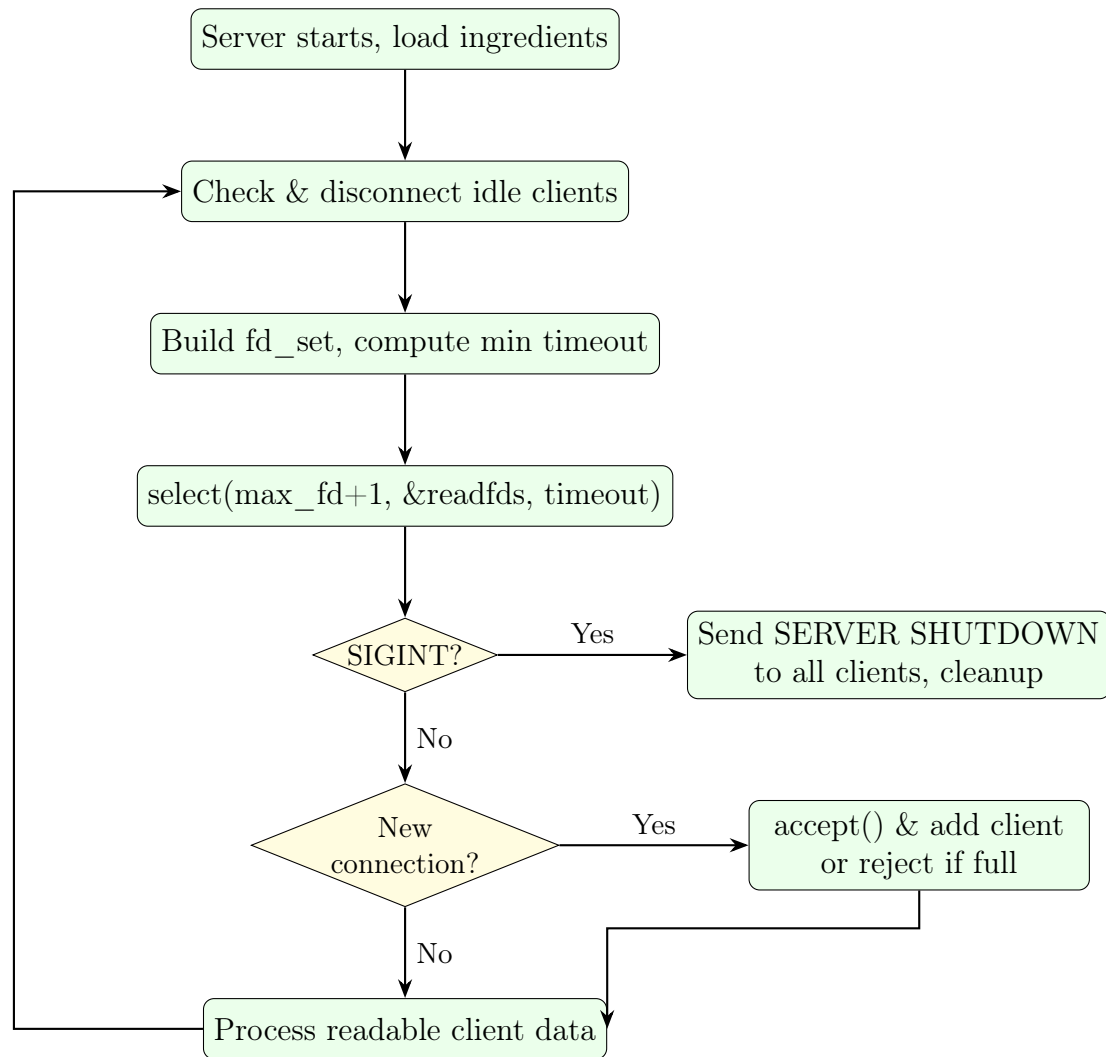


Figure 2: Main server event loop flowchart.

1.3 Idle Timeout Mechanism

Each connected client has a `last_active` timestamp that is updated every time the client sends a command. Before each `select()` call, the server:

1. Iterates over all connected clients.
2. Computes `elapsed = now - last_active` for each client.
3. If `elapsed >= TIMEOUT`, sends `TIMEOUT DISCONNECT` to the client, logs the event, and closes the connection.
4. Computes the minimum remaining time (`TIMEOUT - elapsed`) among all active clients.
5. Passes this minimum as the `struct timeval` timeout to `select()`.

This design ensures idle clients are detected and removed without using `sleep()` or additional threads. The `select()` timeout dynamically adapts based on client activity.

1.4 Maximum Client Enforcement

When the server has reached `max_clients` capacity:

1. The incoming connection is accepted with `accept()` (so the client does not hang).
2. `ERR HOGWARTS_FULL\n` is sent immediately to the new client.
3. The event is logged as `REJECTED` with the file descriptor and IP address.
4. The file descriptor is closed without adding the client to the client array.

This ensures the client receives a meaningful error message rather than experiencing a silent timeout.

2 Data Structures

2.1 Ingredient Storage

Global ingredients are stored in a fixed-size array of `ingredient_t` structs:

```
1 typedef struct {
2     char name[32];    /* e.g., MOONSTONE */
3     int  qty;         /* current global quantity */
4 } ingredient_t;
5
6 static ingredient_t g_ing[256];
7 static int          g_ing_count = 0;
```

At startup, ingredients are loaded from the file specified by the `-s` parameter. Each line must contain a valid uppercase alphanumeric name (max 16 chars) and a positive integer quantity. Lines with invalid format are silently skipped.

2.2 Per-Client State (`client_t`)

Each connected TCP client is represented by a `client_t` structure:

```
1 typedef struct {
2     int      fd;                /* socket file descriptor */
3     char     username[32];      /* registered name */
4     char     type[16];          /* "WIZARD" or "PROFESSOR" */
5     char     line_buf[513];     /* partial read buffer */
6     int      buf_len;           /* current buffer length */
7     int      toolong;           /* flag: line exceeded 512 bytes */
8     time_t   last_active;       /* timestamp of last command */
9     int      enrolled;          /* 1 after successful ENROLL */
10    spell_entry_t spellbook[256]; /* WIZARD only */
11    int      spell_count;        /* entries in spellbook */
12 } client_t;
```

Clients are stored in a dynamically allocated pointer array of size `max_clients`:

```
1 static client_t **g_clients = NULL; /* calloc(max_clients, ...) */
```

A `NULL` entry indicates an empty slot. When a client disconnects, its memory is freed, its slot is set to `NULL`, and the client count is decremented.

2.3 Wizard Spellbook

Each wizard maintains a personal spellbook as an array of `spell_entry_t`:

```
1 typedef struct {
2     char name[32]; /* ingredient name */
3     int  qty;      /* quantity brewed by this wizard */
4 } spell_entry_t;
```

- **BREW:** Increments both the global ingredient quantity and the wizard's spellbook entry.
- **CONSUME:** Checks the wizard's spellbook first. If the wizard has not brewed enough of that ingredient, `ERR INSUFFICIENT_INGREDIENTS` is returned. Otherwise, both global stock and spellbook are decremented.
- **SPELLBOOK:** Lists all spellbook entries with quantity > 0 , or returns `OK SPELLBOOK EMPTY`.

Spellbook data is not persisted – it is lost when the client disconnects.

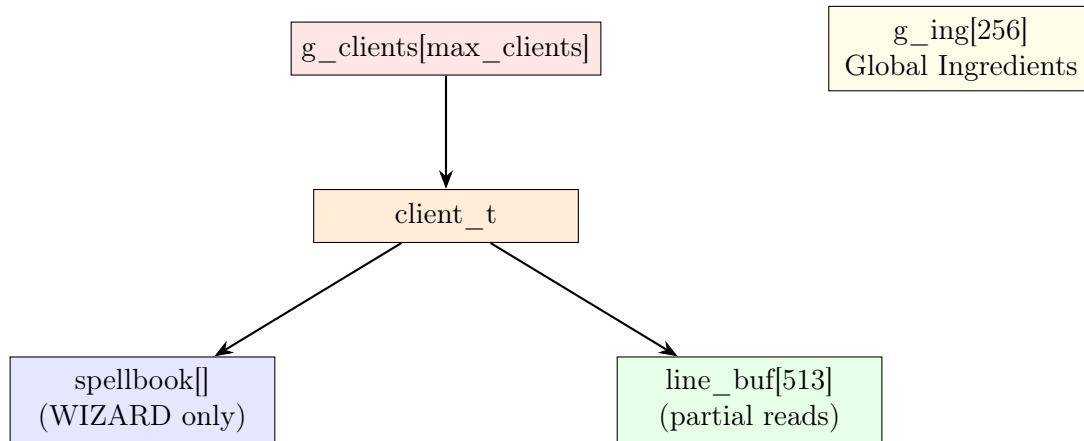


Figure 3: Data structure relationships.

3 Partial Read/Write Handling

3.1 Partial Read Handling

TCP is a byte-stream protocol and does not guarantee message boundaries. A single `read()` call may return a partial line, multiple lines, or a combination. The server handles this with a per-client line buffer (`line_buf`):

1. Data is read from the socket into a temporary buffer.
2. Each byte is appended to the client's `line_buf` one at a time.
3. When a newline character (`\n`) is encountered, the buffer is null-terminated and the complete line is processed.
4. If the buffer reaches 512 bytes without a newline, the `toolong` flag is set, `ERR TOOLONG` is sent, and remaining bytes until the next newline are discarded.
5. After processing, `buf_len` is reset to 0 for the next line.

This design correctly handles:

- Partial reads (line split across multiple `read()` calls).
- Coalesced reads (multiple lines in a single `read()` call).
- Oversized lines (lines exceeding 512 bytes).

The client programs (wizard and professor) use the same line-buffering approach for server responses, accumulating bytes until a newline is found.

3.2 Partial Write Handling

The `send_all()` function ensures complete message delivery by looping on `write()` until all bytes are sent:

```
1 static int send_all(int fd, const char *msg) {
2     size_t len = strlen(msg), sent = 0;
3     while (sent < len) {
4         ssize_t n = write(fd, msg + sent, len - sent);
5         if (n <= 0) return -1;
6         sent += (size_t)n;
7     }
8     return 0;
9 }
```

This function is used in all three programs (server and both clients) whenever data is written to a TCP socket, guaranteeing that partial writes are handled correctly.

4 Signal Handling and Shutdown

4.1 Server Shutdown (Ctrl+C)

The server uses a volatile `sig_atomic_t` flag for safe signal handling:

1. The `SIGINT` handler sets `g_shutdown = 1`. No other operations are performed in the handler (async-signal-safe).
2. The main `select()` loop checks `g_shutdown` at the top of each iteration. If `select()` is interrupted by the signal (`errno == EINTR`), it continues to the next iteration where the flag is checked.
3. Upon detecting shutdown:
 - (a) `SERVER SHUTDOWN\n` is sent to all connected clients.
 - (b) Each client's disconnection is logged with `reason=shutdown`.
 - (c) All client file descriptors are closed and memory is freed.
 - (d) The server listening socket is closed.
 - (e) `SHUTDOWN signal=SIGINT` and `CLEANUP_DONE clients=0` are logged.
 - (f) The log file is closed and all allocated memory is freed.

`SIGPIPE` is set to `SIG_IGN` to prevent the server from crashing when writing to a client that has already disconnected.

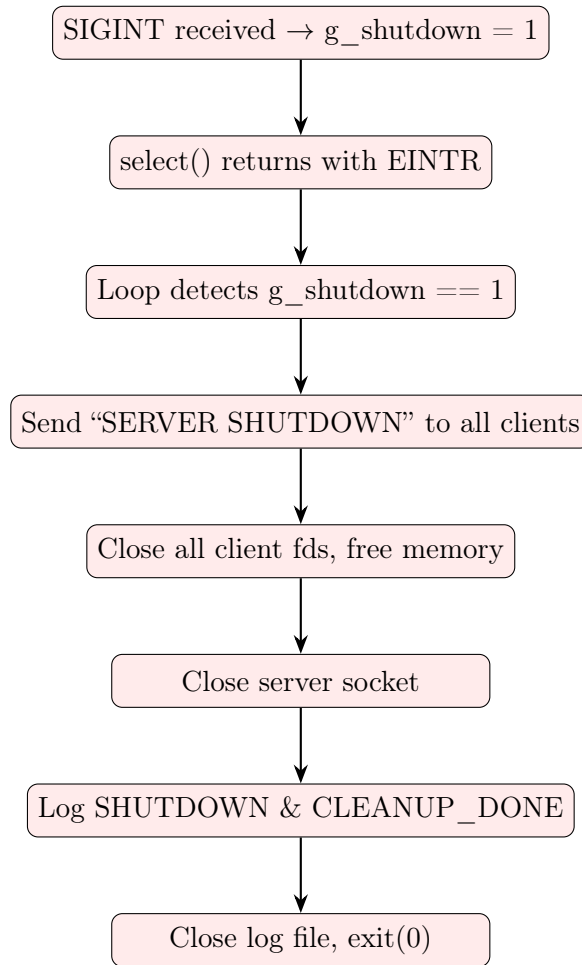


Figure 4: Server shutdown sequence upon `SIGINT`.

4.2 Client Exit (Ctrl+C)

Both wizard and professor clients follow the same shutdown procedure:

1. The `SIGINT` handler sets `g_exit = 1` (async-signal-safe).
2. The main `select()` loop detects the flag (either through `EINTR` or at the loop condition check).
3. The client sends `APPARATE\n` to the server.
4. The client waits briefly (2-second `select()` timeout) for the server's OK `APPARATE` response.
5. The response is printed, `DISCONNECTED reason=APPARATE` is output.
6. The TCP socket is closed and the program exits cleanly.

On the server side, the `APPARATE` command triggers OK `APPARATE`, followed by logging `CLIENT_DISCONNECTED username=X reason=APPARATE` and cleanup of the client's state.

5 Error Handling Summary

The following table summarizes all error conditions handled by the server:

Error Response	Condition	Applies To
ERR ENROLL name_taken	Username already in use	Both
ERR NOT_ENROLLED	Command sent before ENROLL	Both
ERR UNKNOWN <cmd>	Unrecognized command	Both
ERR TOOLONG	Line exceeds 512 bytes	Both
ERR HOGWARTS_FULL	Server at max_clients capacity	New conn.
ERR UNAUTHORIZED	Command not allowed for type	Both
ERR UNKNOWN_INGREDIENT	Ingredient not in inventory	Wizard/Prof.
ERR INSUFFICIENT_INGREDIENTS	Not enough in spellbook	Wizard
TIMEOUT DISCONNECT	Client idle $\geq$ TIMEOUT sec	Both
SERVER SHUTDOWN	Server received SIGINT	Both

Table 1: Complete list of error responses.

6 Compilation and Execution

All three programs are compiled using the provided Makefile:

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11

make          # builds hogwarts, wizard, professor
make clean    # removes binaries
```

Example execution:

```
# Terminal 1 - Server
./hogwarts -p 5050 -s ingredients.txt -l server.log -n 4 -t 30

# Terminal 2 - Wizard
./wizard 127.0.0.1 5050 hermione

# Terminal 3 - Professor
./professor 127.0.0.1 5050 dumbledore
```

7 Test Scenarios

The following four mandatory test scenarios were executed. Uncut terminal screenshots are included below. Each scenario was run with a fresh server instance.

7.1 Scenario 1: Error Cases

Server: `./hogwarts -p 5050 -s ingredients.txt -l server.log -n 4 -t 30`

Tests performed:

- Duplicate username enrollment $\rightarrow$ ERR ENROLL name_taken
- Command before ENROLL $\rightarrow$ ERR NOT_ENROLLED

- Unknown command (FLY BROOMSTICK) → ERR UNKNOWN
- Professor BREW/CONSUME → ERR UNAUTHORIZED
- Wizard INSPECT/SCROLL → ERR UNAUTHORIZED
- Clean Ctrl+C disconnect for both clients

```

root@b8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 300
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 300s

root@b8f5484091ea:~/hw6# ./professor 127.0.0.1 5000
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
> BREW MOONSTONE 5
[PROFESSOR dumbledore] SENT BREW MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
> CONSUME MOONSTONE 5
[PROFESSOR dumbledore] SENT CONSUME MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
> ^C
[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
root@b8f5484091ea:~/hw6#

root@b8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
> FLY BROOMSTICK
[WIZARD hermione] SENT FLY BROOMSTICK
[WIZARD hermione] RECEIVED ERR UNKNOWN FLY BROOMSTICK
> INSPECT MOONSTONE
[WIZARD hermione] SENT INSPECT MOONSTONE
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
> SCROLL
[WIZARD hermione] SENT SCROLL
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
> ^C
[WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
root@b8f5484091ea:~/hw6#

root@b8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED ERR ENROLL name_taken
>

```

Figure 5: Scenario 1

7.2 Scenario 2: Normal Wizard and Professor Operations

Server: `./hogwarts -p 5050 -s ingredients.txt -l server.log -n 4 -t 30`

Tests performed:

- BREW MOONSTONE 10, BREW UNICORN_HAIR 15
- SPELLBOOK after each brew
- CONSUME MOONSTONE 5 (success), CONSUME UNICORN_HAIR 20 (insufficient)
- BREW BUTTERBEER 5 → ERR UNKNOWN_INGREDIENT
- INSPECT, SCROLL, ROSTER from professor
- INSPECT BUTTERBEER → ERR UNKNOWN_INGREDIENT

```

root@8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 30s
% root@8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
Hogwarts is ready. Port: 5000 | Max Clients: 4 | Timeout: 30s

root@8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
> BREW MOONSTONE 10
[WIZARD harry] SENT BREW MOONSTONE 10
[WIZARD harry] RECEIVED OK BREW MOONSTONE 10 110
> SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10
> BREW UNICORN_HAIR 15
[WIZARD harry] SENT BREW UNICORN_HAIR 15
[WIZARD harry] RECEIVED OK BREW UNICORN_HAIR 15 35
> SPELLBOOK
[WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10,UNICORN_HAIR:15
> CONSUME MOONSTONE 5
[WIZARD harry] SENT CONSUME MOONSTONE 5
[WIZARD harry] RECEIVED OK CONSUME MOONSTONE 5 105
> CONSUME UNICORN_HAIR 20
[WIZARD harry] SENT CONSUME UNICORN_HAIR 20
[WIZARD harry] RECEIVED ERR INSUFFICIENT_INGREDIENTS
> BREW BUTTERBEER 5
[WIZARD harry] SENT BREW BUTTERBEER 5
[WIZARD harry] RECEIVED ERR UNKNOWN_INGREDIENT
> [WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=timeout
root@8f5484091ea:~/hw6# ^C
root@8f5484091ea:~/hw6#

root@8f5484091ea:~/hw6# ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
> [PROFESSOR mcgonagall] RECEIVED TIMEOUT DISCONNECT
[PROFESSOR mcgonagall] DISCONNECTED reason=timeout
root@8f5484091ea:~/hw6# ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
> INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] SENT INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN INSPECT DRAGON_SCALE
> SCROLL
[PROFESSOR mcgonagall] SENT SCROLL
[PROFESSOR mcgonagall] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:50,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:35
> ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER harry,mcgonagall
> INSPECT BUTTERBEER
[PROFESSOR mcgonagall] SENT INSPECT BUTTERBEER
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN_INGREDIENT
> ^C
[PROFESSOR mcgonagall] SENT APPARATE
[PROFESSOR mcgonagall] RECEIVED OK APPARATE
[PROFESSOR mcgonagall] DISCONNECTED reason=APPARATE
root@8f5484091ea:~/hw6#

```

Figure 6: Scenario 2

7.3 Scenario 3: Max Clients and Dynamic Connections

Server: `./hogwarts -p 5050 -s ingredients.txt -l server.log -n 3 -t 60`

Tests performed:

- 3 clients fill the server (harry, ron, dumbledore)
- 4th and 5th clients → `ERR HOGWARTS_FULL`
- harry disconnects, freeing a slot
- neville connects successfully into freed slot
- BREW, ROSTER commands verified
- Orderly Ctrl+C shutdown of remaining clients

7.4 Scenario 4: Idle Timeout and Server Shutdown

Server: `./hogwarts -p 5050 -s ingredients.txt -l server.log -n 6 -t 10`

Tests performed:

- hermione sends commands with 3-second gaps (stays alive)
- harry sends nothing for 10 seconds → `TIMEOUT DISCONNECT`
- Multiple new clients connect after timeout
- Server Ctrl+C → `SERVER SHUTDOWN` sent to all clients
- `ps aux | grep hogwarts` confirms no zombie processes
- `cat server.log` shows complete log with all events

```

root@8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 3 -t 60
Hogwarts is ready. Port: 5000 | Max Clients: 3 | Timeout: 60s
[]

root@8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
> ^C
[WIZARD ron] SENT APPARATE
[WIZARD ron] RECEIVED OK APPARATE
[WIZARD ron] DISCONNECTED reason=APPARATE
❖ root@8f5484091ea:~/hw6# []

root@8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
> ^C
[WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
❖ root@8f5484091ea:~/hw6# []

root@8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 neville
[WIZARD neville] CONNECTED server=127.0.0.1:5000
[WIZARD neville] RECEIVED OK ENROLL neville
> BREW MOONSTONE 5
[WIZARD neville] SENT BREW MOONSTONE 5
[WIZARD neville] RECEIVED OK BREW MOONSTONE 5 105
> ^C
[WIZARD neville] SENT APPARATE
[WIZARD neville] RECEIVED OK APPARATE
[WIZARD neville] DISCONNECTED reason=APPARATE
❖ root@8f5484091ea:~/hw6# []

root@8f5484091ea:~/hw6# ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
> ROSTER
[PROFESSOR dumbledore] SENT ROSTER
[PROFESSOR dumbledore] RECEIVED OK ROSTER neville,ron,dumbledore
> ^C
[PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
❖ root@8f5484091ea:~/hw6# []

root@8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 voldemort
[WIZARD voldemort] CONNECTED server=127.0.0.1:5000
[WIZARD voldemort] RECEIVED ERR HOGWARTS_FULL
>
[WIZARD voldemort] DISCONNECTED reason=shutdown
❖ root@8f5484091ea:~/hw6# []

root@8f5484091ea:~/hw6# ./professor 127.0.0.1 5000 Snape
[PROFESSOR Snape] CONNECTED server=127.0.0.1:5000
[PROFESSOR Snape] RECEIVED ERR HOGWARTS_FULL
>
[PROFESSOR Snape] DISCONNECTED reason=shutdown
❖ root@8f5484091ea:~/hw6# []

```

Figure 7: Scenario 3

```
bash hw6 X
root@b8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10
Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 10s
root@b8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 30
Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 30s
^Croot@b8f5484091ea:~/hw6# ^C
root@b8f5484091ea:~/hw6# ps aux | grep hogwarts
bash: ps: command not found
root@b8f5484091ea:~/hw6# ps aux | grep hogwarts
bash: ps: command not found
root@b8f5484091ea:~/hw6# cat server.log
[2026-05-20 15:38:14] SERVER_STARTED port=6000 max_clients=4 timeout=300 ingredients=5
[2026-05-20 15:38:29] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 15:38:29] ENROLL_username=hermione type=WIZARD fd=5
[2026-05-20 15:38:42] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 15:38:42] ENROLL_username=dumbledore type=PROFESSOR fd=6
[2026-05-20 15:38:54] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-20 15:42:03] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-20 15:42:09] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 15:42:36] CLIENT_DISCONNECTED username=(unknown) reason=shutdown
[2026-05-20 15:42:36] SHUTDOWN signal=SIGINT
[2026-05-20 15:42:36] CLEANUP_DONE clients=0
[2026-05-20 15:44:39] SERVER_STARTED port=6000 max_clients=4 timeout=30 ingredients=5
[2026-05-20 15:44:56] SHUTDOWN signal=SIGINT
[2026-05-20 15:44:56] CLEANUP_DONE clients=0
[2026-05-20 15:45:03] SERVER_STARTED port=6000 max_clients=4 timeout=300 ingredients=5
[2026-05-20 15:45:36] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 15:45:36] ENROLL_username=dumbledore type=PROFESSOR fd=5

root@b8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5000
[WIZARD hermione] RECEIVED OK ENROLL hermione
> BREW MOONSTONE 5
[WIZARD hermione] SENT BREW MOONSTONE 5
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 5 105
> SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5
> BREW DRAGON_SCALE 2
[WIZARD hermione] SENT BREW DRAGON_SCALE 2
[WIZARD hermione] RECEIVED OK BREW DRAGON_SCALE 2 52
> SPELLBOOK
[WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
> [WIZARD hermione] RECEIVED TIMEOUT DISCONNECT
[WIZARD hermione] DISCONNECTED reason=timeout
root@b8f5484091ea:~/hw6#

root@b8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5000
[WIZARD harry] RECEIVED OK ENROLL harry
> [WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=timeout
root@b8f5484091ea:~/hw6# ./wizard 127.0.0.1 5000 ron
[WIZARD ron] CONNECTED server=127.0.0.1:5000
[WIZARD ron] RECEIVED OK ENROLL ron
> BREW UNICORN_HAIR 5
[WIZARD ron] SENT BREW UNICORN_HAIR 5
[WIZARD ron] RECEIVED OK BREW UNICORN_HAIR 5 25
> [WIZARD ron] RECEIVED SERVER SHUTDOWN
[WIZARD ron] DISCONNECTED reason=shutdown
root@b8f5484091ea:~/hw6#

root@b8f5484091ea:~/hw6# ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5000
[PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
> ROSTER
[PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER ron,mcgonagall,dumbledore
> [PROFESSOR mcgonagall] RECEIVED SERVER SHUTDOWN
[PROFESSOR mcgonagall] DISCONNECTED reason=shutdown
root@b8f5484091ea:~/hw6#

root@b8f5484091ea:~/hw6# ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5000
[PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
> SCROLL
[PROFESSOR dumbledore] SENT SCROLL
[PROFESSOR dumbledore] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:52,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:25
> [PROFESSOR dumbledore] RECEIVED SERVER SHUTDOWN
[PROFESSOR dumbledore] DISCONNECTED reason=shutdown
root@b8f5484091ea:~/hw6#
```

Figure 8: Scenario 4 (Extended version at the below)

```

root@b8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10

Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 10s
root@b8f5484091ea:~/hw6# ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 30
Hogwarts is ready. Port: 5000 | Max Clients: 6 | Timeout: 30s
^C
root@b8f5484091ea:~/hw6# ps aux | grep hogwarts
root      77132  0.0  0.0   2908  1588 pts/1    S+   14:21   0:00 grep hogwarts
root@b8f5484091ea:~/hw6# cat server.log
[2026-05-20 15:38:14] SERVER_STARTED port=6000 max_clients=4 timeout=300 ingredients=5
[2026-05-20 15:38:29] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 15:38:29] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 15:38:42] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 15:38:42] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 15:38:54] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-20 15:42:03] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-20 15:42:09] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 15:42:36] CLIENT_DISCONNECTED username=(unknown) reason=shutdown
[2026-05-20 15:42:36] SHUTDOWN signal=SIGINT
[2026-05-20 15:42:36] CLEANUP_DONE clients=0
[2026-05-20 15:44:39] SERVER_STARTED port=6000 max_clients=4 timeout=30 ingredients=5
[2026-05-20 15:44:56] SHUTDOWN signal=SIGINT
[2026-05-20 15:44:56] CLEANUP_DONE clients=0
[2026-05-20 15:45:03] SERVER_STARTED port=6000 max_clients=4 timeout=300 ingredients=5
[2026-05-20 15:45:36] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 15:45:36] ENROLL username=dumbledore type=PROFESSOR fd=5
[2026-05-20 15:45:40] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 15:45:44] SHUTDOWN signal=SIGINT
[2026-05-20 15:45:44] CLEANUP_DONE clients=0
[2026-05-20 15:47:37] SERVER_STARTED port=6000 max_clients=4 timeout=300 ingredients=5
[2026-05-20 15:47:44] SHUTDOWN signal=SIGINT
[2026-05-20 15:47:44] CLEANUP_DONE clients=0
[2026-05-20 15:50:39] SERVER_STARTED port=5050 max_clients=4 timeout=300 ingredients=5
[2026-05-20 15:50:47] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 15:50:47] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 15:50:55] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 15:50:55] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 15:52:05] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-20 15:52:12] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 15:52:25] SHUTDOWN signal=SIGINT
[2026-05-20 15:52:25] CLEANUP_DONE clients=0
[2026-05-20 15:55:27] SERVER_STARTED port=6000 max_clients=3 timeout=600 ingredients=5
[2026-05-20 15:57:00] SHUTDOWN signal=SIGINT
[2026-05-20 15:57:00] CLEANUP_DONE clients=0
[2026-05-20 15:58:14] SERVER_STARTED port=6000 max_clients=3 timeout=600 ingredients=5
[2026-05-20 15:58:39] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 15:58:39] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 15:58:45] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 15:58:45] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 15:58:53] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-20 15:58:53] ENROLL username=dumbledore type=PROFESSOR fd=7
[2026-05-20 15:58:59] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 15:58:59] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 15:59:07] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 15:59:07] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL

```

Figure 9: Scenario 4 - Extended Hogwarts terminal[Part1]

```

[2026-05-20 15:59:26] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 15:59:26] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 15:59:35] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-20 16:00:21] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-20 16:00:29] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 13:52:37] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredients=5
[2026-05-20 13:53:00] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 13:53:00] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 13:53:12] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 13:53:12] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 13:54:13] [SERVER] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-20 13:54:14] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 13:58:29] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 13:58:29] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 13:58:38] [SERVER] SERVER_STARTED port=5000 max_clients=4 timeout=30 ingredients=5
[2026-05-20 13:58:44] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 13:58:44] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 13:58:51] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 13:58:51] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-20 13:58:59] [SERVER] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-20 13:59:11] [SERVER] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-20 13:59:21] [SERVER] TIMEOUT username=mcgonagall fd=6 elapsed=30s
[2026-05-20 13:59:21] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=timeout
[2026-05-20 13:59:23] [SERVER] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-20 13:59:44] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 13:59:44] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-20 13:59:58] [SERVER] SCROLL professor=mcgonagall ingredients=5
[2026-05-20 14:00:03] [SERVER] ROSTER professor=mcgonagall clients=2
[2026-05-20 14:00:05] [SERVER] TIMEOUT username=harry fd=5 elapsed=30s
[2026-05-20 14:00:05] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 14:00:18] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
[2026-05-20 14:01:49] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 14:01:49] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 14:02:20] [SERVER] SERVER_STARTED port=5000 max_clients=3 timeout=60 ingredients=5
[2026-05-20 14:02:27] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 14:02:27] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 14:02:39] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-20 14:02:44] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 14:02:44] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 14:04:00] [SERVER] SERVER_STARTED port=5000 max_clients=3 timeout=60 ingredients=5
[2026-05-20 14:04:13] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 14:04:13] [SERVER] ENROLL username=harry type=WIZARD fd=5
[2026-05-20 14:04:21] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 14:04:21] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 14:04:28] [SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-20 14:04:28] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=7
[2026-05-20 14:04:34] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 14:04:34] [SERVER] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 14:04:40] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 14:04:40] [SERVER] REJECTED fd=8 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-20 14:04:45] [SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-20 14:04:55] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 14:04:55] [SERVER] ENROLL username=neville type=WIZARD fd=5
[2026-05-20 14:05:03] [SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105

```

Figure 10: Scenario 4 - Extended Hogwarts terminal[Part2]

```

[2026-05-20 14:05:10] [SERVER] ROSTER professor=dumbledore clients=3
[2026-05-20 14:05:15] [SERVER] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-20 14:05:20] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-20 14:05:22] [SERVER] CLIENT_DISCONNECTED username=neville reason=APPARATE
[2026-05-20 14:11:57] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 14:11:57] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 14:13:08] [SERVER] SERVER_STARTED port=5000 max_clients=6 timeout=10 ingredients=5
[2026-05-20 14:13:09] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 14:13:09] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 14:13:11] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 14:13:11] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-20 14:13:13] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 14:13:21] [SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-20 14:13:21] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 14:13:27] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-20 14:13:42] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 14:13:42] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-20 14:13:45] [SERVER] TIMEOUT username=hermione fd=5 elapsed=10s
[2026-05-20 14:13:45] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-20 14:13:49] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 14:13:49] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=5
[2026-05-20 14:13:52] [SERVER] TIMEOUT username=dumbledore fd=6 elapsed=10s
[2026-05-20 14:13:52] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=timeout
[2026-05-20 14:13:59] [SERVER] TIMEOUT username=mcgonagall fd=5 elapsed=10s
[2026-05-20 14:13:59] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=timeout
[2026-05-20 14:14:12] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 14:14:12] [SERVER] CLEANUP_DONE clients=0
[2026-05-20 14:16:49] [SERVER] SERVER_STARTED port=5000 max_clients=6 timeout=30 ingredients=5
[2026-05-20 14:16:51] [SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-20 14:16:51] [SERVER] ENROLL username=hermione type=WIZARD fd=5
[2026-05-20 14:16:53] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 14:16:53] [SERVER] ENROLL username=harry type=WIZARD fd=6
[2026-05-20 14:17:06] [SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-20 14:17:23] [SERVER] TIMEOUT username=harry fd=6 elapsed=30s
[2026-05-20 14:17:23] [SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-20 14:17:37] [SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-20 14:17:45] [SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-20 14:17:45] [SERVER] ENROLL username=ron type=WIZARD fd=6
[2026-05-20 14:17:50] [SERVER] CLIENT_CONNECTED fd=7 ip=127.0.0.1
[2026-05-20 14:17:50] [SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=7
[2026-05-20 14:17:51] [SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-20 14:17:51] [SERVER] ENROLL username=dumbledore type=PROFESSOR fd=8
[2026-05-20 14:17:58] [SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-20 14:18:09] [SERVER] SCROLL professor=dumbledore ingredients=5
[2026-05-20 14:18:09] [SERVER] TIMEOUT username=hermione fd=5 elapsed=30s
[2026-05-20 14:18:09] [SERVER] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-20 14:18:14] [SERVER] ROSTER professor=mcgonagall clients=3
[2026-05-20 14:18:18] [SERVER] CLIENT_DISCONNECTED username=ron reason=shutdown
[2026-05-20 14:18:18] [SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=shutdown
[2026-05-20 14:18:18] [SERVER] CLIENT_DISCONNECTED username=dumbledore reason=shutdown
[2026-05-20 14:18:18] [SERVER] SHUTDOWN signal=SIGINT
[2026-05-20 14:18:18] [SERVER] CLEANUP_DONE clients=0

```

Figure 11: Scenario 4 - Extended Hogwarts terminal[Part3]

8 Conclusion

This homework implemented a complete multi-client TCP server/client system using `select()`-based I/O multiplexing in C. The server successfully handles concurrent client connections in a single thread, enforces role-based command authorization, manages ingredient inventory with per-wizard spellbooks, implements idle timeout without threads or `sleep()`, and performs clean shutdown upon SIGINT. All error conditions specified in the assignment are properly handled and return the correct error messages.